LSTM Practice on NVIDIA 60 days price forecast



24 de agosto de 2026

7.7 LSTM forecasting Max score: 1

1. Build and evaluate a Long Short-Term Memory (LSTM) neural network to forecast future values of a time series dataset. Choose a univariate or multivariate time series dataset (e.g., stock prices, energy consumption, weather data). Ensure the dataset has sufficient observations (at least 500 data points for meaningful training).
2. Handle missing values and normalize/scale the data (e.g., MinMaxScaler). Split the data into training, validation, and test sets.
3. Implement an LSTM-based architecture using a deep learning framework (TensorFlow/Keras or PyTorch). Experiment with hyperparameters (number of units, batch size, epochs).
4. Train the model on the training set and validate on the validation set. Evaluate performance on the test set using metrics such as RMSE, MAE, and MAPE. Compare predictions with actual values.
5. Plot original time series and actual vs predicted values for the test set
6. Interpret the results and discuss how well the model captures trends and seasonality

In [21]:

```
# Req. 1: Downloading Meta Platforms (META) stock price data from 2000 to 2026

!pip install yfinance

import yfinance as yf

# Download historical Meta stock data
df = yf.download(
    "META",
    start="2000-01-01",
    end="2026-08-24"
)

# Save data to CSV file
df.to_csv(r"S:\Python_Selfstudy\META.csv")

# Display first 5 rows
print(df.head())
```

Out [21]:

```
[notice] A new release of pip is available: 26.0.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
[**********************100%**********************]  1 of 1 completed
Requirement already satisfied: yfinance in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (1.5.2)
Requirement already satisfied: pandas>=1.3.0 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
yfinance) (2.2.3)
Requirement already satisfied: numpy>=1.16.5 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
yfinance) (2.1.3)
Requirement already satisfied: requests>=2.31 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
yfinance) (2.32.3)
Requirement already satisfied: multitasking>=0.0.7 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
yfinance) (0.0.13)
Requirement already satisfied: platformdirs>=2.0.0 in C:\Users\ROWTECH\AppData\Roaming\Python\Python313\site-packages (from yfinance)
(4.3.6)
Requirement already satisfied: pytz>=2022.5 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
yfinance) (2024.2)
Requirement already satisfied: peewee>=3.16.2 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
yfinance) (4.2.6)
Requirement already satisfied: beautifulsoup4>=4.11.1 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages
(from yfinance) (4.12.3)
Requirement already satisfied: curl_cffi>=0.15 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
yfinance) (0.15.0)
Requirement already satisfied: protobuf>=3.19.0 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
yfinance) (7.35.1)
Requirement already satisfied: websockets>=13.0 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
yfinance) (16.1.1)
Requirement already satisfied: soupsieve>1.2 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
beautifulsoup4>=4.11.1->yfinance) (2.6)
Requirement already satisfied: cffi>=2.0.0 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
curl_cffi>=0.15->yfinance) (2.0.0)
Requirement already satisfied: certifi>=2024.2.2 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
```

```
curl_cffi>=0.15->yfinance) (2025.8.3)
Requirement already satisfied: rich in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
curl_cffi>=0.15->yfinance) (15.0.0)
Requirement already satisfied: pycparser in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
cffi>=2.0.0->curl_cffi>=0.15->yfinance) (2.22)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\ROWTECH\AppData\Roaming\Python\Python313\site-packages (from
pandas>=1.3.0->yfinance) (2.9.0.post0)
Requirement already satisfied: tzdata>=2022.7 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
pandas>=1.3.0->yfinance) (2024.2)
Requirement already satisfied: six>=1.5 in C:\Users\ROWTECH\AppData\Roaming\Python\Python313\site-packages (from python-
dateutil>=2.8.2->pandas>=1.3.0->yfinance) (1.16.0)
Requirement already satisfied: charset-normalizer<4,>=2 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages
(from requests>=2.31->yfinance) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
requests>=2.31->yfinance) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
requests>=2.31->yfinance) (2.5.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages
(from rich->curl_cffi>=0.15->yfinance) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in C:\Users\ROWTECH\AppData\Roaming\Python\Python313\site-packages (from rich-
>curl_cffi>=0.15->yfinance) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in C:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
markdown-it-py>=2.2.0->rich->curl_cffi>=0.15->yfinance) (0.1.2)
Price           Close       High        Low       Open     Volume
Ticker           META       META       META       META       META
Date
2012-05-18  37.897202  44.608268  37.669204  41.683948  573576400
2012-05-21  33.733765  36.340871  32.712732  36.212002  168192700
2012-05-22  30.730143  33.297597  30.670665  32.326128  101786600
2012-05-23  31.721436  32.217083  31.087007  31.096921   73600000
2012-05-24  32.742466  32.920899  31.493436  32.663164   50237200
```

In [22]:

```
#Req. 2 Handle missing values and normalize/scale the data (e.g., MinMaxScaler). Split the data into training, validation, and tes

import pandas as pd
import numpy as np

from sklearn.preprocessing import MinMaxScaler

# Load data
df = pd.read_csv("META.csv")
print(df.head())


#Colum headers containing 3 cells, so we need to rename them to 1 cell each
```

Out [22]:

```
        Price              Close               High                Low  \
0      Ticker               META               META               META
1        Date                NaN                NaN                NaN
2  2012-05-18  37.89720153808594   44.60826810447062  37.669204177108526
3  2012-05-21   33.7337646484375  36.340871326058476  32.712732099058826
4  2012-05-22  30.73014259338379   33.29759659357449  30.670665427450363

                Open     Volume
0               META       META
1                NaN        NaN
2   41.68394755021293  573576400
3  36.212001928669224  168192700
4   32.32612802343342  101786600
```

In [23]:

```
#date formating

import pandas as pd

df = pd.read_csv(
    "META.csv",
    skiprows=[1]  # skip the Ticker row
)
```

```
print(df.head())
```

Out [23]:

```
        Price      Close       High        Low       Open       Volume
0        Date        NaN        NaN        NaN        NaN          NaN
1  2012-05-18  37.897202  44.608268  37.669204  41.683948  573576400.0
2  2012-05-21  33.733765  36.340871  32.712732  36.212002  168192700.0
3  2012-05-22  30.730143  33.297597  30.670665  32.326128  101786600.0
4  2012-05-23  31.721436  32.217083  31.087007  31.096921   73600000.0
```

In [24]:

```
# Last 10 rows
12
print("Last 10 rows:")
13
print(df.tail(10))
```

Out [24]:

```
Last 10 rows:
           Price       Close        High         Low        Open      Volume
3576  2026-08-10  594.919983  608.210022  592.030029  599.130005  14967300.0
3577  2026-08-11  599.119995  612.429993  593.400024  598.000000  13146900.0
3578  2026-08-12  578.849976  604.500000  578.210022  601.070007  16186900.0
3579  2026-08-13  594.969971  595.849976  579.390015  580.710022  11164200.0
3580  2026-08-14  589.849976  601.859985  589.289978  596.979980   8758000.0
3581  2026-08-17  568.969971  590.239990  564.750000  589.750000  17225600.0
3582  2026-08-18  543.669983  560.659973  543.210022  557.479980  27094900.0
3583  2026-08-19  546.030029  554.070007  537.270020  541.340027  16971700.0
3584  2026-08-20  545.830017  549.989990  539.309998  546.950012  13872200.0
3585  2026-08-21  549.900024  553.869995  543.229980  546.890015  13663800.0
```

In [25]:

```
#date formating
# still the extra row

import pandas as pd

df = pd.read_csv("META.csv", skiprows=[1])

# Remove the first row containing 'Date'
df = df.iloc[1:]

# Rename Price column to Date
df.rename(columns={"Price": "Date"}, inplace=True)

# Convert Date to datetime
df["Date"] = pd.to_datetime(df["Date"])

# Set Date as index
df.set_index("Date", inplace=True)

print(df.head())
print(df.info())
```

Out [25]:

```
                Close       High        Low       Open       Volume
Date
2012-05-18  37.897202  44.608268  37.669204  41.683948  573576400.0
2012-05-21  33.733765  36.340871  32.712732  36.212002  168192700.0
2012-05-22  30.730143  33.297597  30.670665  32.326128  101786600.0
2012-05-23  31.721436  32.217083  31.087007  31.096921   73600000.0
2012-05-24  32.742466  32.920899  31.493436  32.663164   50237200.0
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 3585 entries, 2012-05-18 to 2026-08-21
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Close   3585 non-null   float64
 1   High    3585 non-null   float64
 2   Low     3585 non-null   float64
 3   Open    3585 non-null   float64
 4   Volume  3585 non-null   float64
```

```
dtypes: float64(5)
memory usage: 168.0 KB
None
```

In [26]:

```
# checking the null values & good to see, there is no null values
print(df.isnull().sum())
```

Out [26]:

```
Close     0
High      0
Low       0
Open      0
Volume    0
dtype: int64
```

In [27]:

```
#normalize/scale the data (e.g., MinMaxScaler)
    #LSTM neural networks generally learn better when input values are in a small range, such as 0 to 1. Values are easier for the

from sklearn.preprocessing import MinMaxScaler
    ### Scikit-Learn contains many ready-made machine learning tools.

close_data = df[["Close"]]
    ##We want a separate variable containing only the Close prices.

scaler = MinMaxScaler(feature_range=(0,1))
    ## Stores the MinMaxScaler object under name"scaler'
scaled_data = scaler.fit_transform(close_data)
    ##We need somewhere to keep the transformed data.
print(scaled_data[:5])
```

Out [27]:

```
[[0.02639698]
 [0.02098882]
 [0.01708722]
 [0.01837487]
 [0.01970116]]
```

In [28]:

```
#Split the data into training, validation, and test sets.
#Reason: If you let the LSTM see all 2404 days during training, then later asking "How good are your predictions?" would be unfair
#So, give the model some % of data and check how it performs(Training), is the model learning correctly(Validation) here actual da

# Calculate 70% of the data for training

train_size = int(len(scaled_data) * 0.70)
# len(scaled_data) gets total number of rows.
# * 0.70 means 70% of the data.
# int() converts the result to a whole number.

# Calculate 15% of the data for validation

val_size = int(len(scaled_data) * 0.15)
# We reserve 15% of the data for validation.
# Validation data helps check model performance while training.

# Create the training dataset

train_data = scaled_data[:train_size]
# Start from the first row and take data up to train_size.
# This data will be used to teach the LSTM model.

# Create the validation dataset

val_data = scaled_data[train_size:train_size+val_size]
# Start where training data ends.
```

```
# Take the next 15% of the data.
# This data is used to evaluate the model during training.

# Create the test dataset

test_data = scaled_data[train_size+val_size:]
# Start where validation data ends.
# Take all remaining rows.
# This data is kept unseen until the final evaluation.

# Print the number of rows in each dataset

print("Training samples:", len(train_data))
# Shows how many rows are in the training set.

print("Validation samples:", len(val_data))
# Shows how many rows are in the validation set.

print("Test samples:", len(test_data))
# Shows how many rows are in the test set.
```

Out [28]:

```
Training samples: 2509
Validation samples: 537
Test samples: 539
```

Req. 3: Implement an LSTM-based architecture using TensorFlow/Keras or PyTorch. Experiment with hyperparameters (number of units, batch size, epochs).

So, after loading, cleaning, scalled and spilt data, Now you need to create the brain that will learn NVIDIA price patterns.

That brain is the LSTM model. (Implement an LSTM-based architecture)

```
LSTM = the machine (model)
TensorFlow / Keras / PyTorch = the toolbox used to build the machine

Hyperparameter:
    Units

    → Number of memory cells in LSTM.

    → More units = more learning capacity.



    Batch Size

    → Number of training samples processed at a time.



    Epochs

    → Number of times the model sees the entire training dataset.
```

In [29]:

```
#build the LSTM model.

from tensorflow.keras.models import Sequential
# Import Sequential, which allows us to build the neural network layer by layer.

from tensorflow.keras.layers import LSTM, Dense, Dropout
# Import the layers we need: LSTM for learning patterns, Dense for output, Dropout to reduce overfitting.

model = Sequential()
# Create an empty neural network model.

model.add(LSTM(64, return_sequences=True, input_shape=(60,1)))
```

```
# Add the first LSTM layer with 64 memory units, looking at 60 previous days and 1 feature (Close price).

model.add(Dropout(0.2))
# Randomly ignore 20% of neurons during training to help prevent overfitting.

model.add(LSTM(32))
# Add a second LSTM layer with 32 memory units to learn more complex patterns.

model.add(Dropout(0.2))
# Again ignore 20% of neurons during training.

model.add(Dense(1))
# Add the output layer that predicts one value: the next Close price.

model.compile(
    optimizer="adam",
    loss="mean_squared_error"
)
# Configure the model: Adam updates weights efficiently and MSE measures prediction error.

model.summary()
# Display the structure of the model.
```

Out [29]:

```
c:\Users\ROWTECH\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\rnn\rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm_2 (LSTM)	(None, 60, 64)	16,896
dropout_2 (Dropout)	(None, 60, 64)	0
lstm_3 (LSTM)	(None, 32)	12,416
dropout_3 (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 1)	33

Total params: 29,345 (114.63 KB)
Trainable params: 29,345 (114.63 KB)
Non-trainable params: 0 (0.00 B)

REQ: 4 Train the model on the training set and validate on the validation set. Evaluate performance on the test set using metrics such as RMSE, MAE, and MAPE. Compare predictions with actual values.

We just buit a model but it has no learn anything yet Use the training data to teach the LSTM model, while using the validation data to monitor whether the model is learning correctly and not overfitting. Overfitting means memorizing and give answer only when you see the same question in exam, NOT leanring the concept and solving accordingly

RMSE, MAE, and MAPE evaluation metrics used after training to measure how good your predictions are

In [30]:

```
#Training Steps

import numpy as np
# Import NumPy for working with arrays.

def create_dataset(data, time_step=60):
# Create a function to convert a time series into LSTM input-output sequences.

    X, y = [], []
# Create empty lists to store inputs (X) and target values (y).

    for i in range(len(data) - time_step):
# Loop through the dataset and create sequences of 60 days.
```

```
        X.append(data[i:(i + time_step), 0])
# Store the previous 60 days as one input sample.

        y.append(data[i + time_step, 0])
# Store the value immediately after those 60 days as the target.

    return np.array(X), np.array(y)
# Convert the lists into NumPy arrays and return them.


X_train, y_train = create_dataset(train_data)
# Create training inputs (X_train) and training targets (y_train).

X_val, y_val = create_dataset(val_data)
# Create validation inputs (X_val) and validation targets (y_val).

X_test, y_test = create_dataset(test_data)
# Create test inputs (X_test) and test targets (y_test).


X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
# Reshape training inputs into the format required by LSTM: (samples, time steps, features).

X_val = X_val.reshape(X_val.shape[0], X_val.shape[1], 1)
# Reshape validation inputs into the format required by LSTM.

X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)
# Reshape test inputs into the format required by LSTM.


print(X_train.shape)
# Display the shape of the training input data.

print(y_train.shape)
# Display the shape of the training target data.

print(X_val.shape)
# Display the shape of the validation input data.

print(y_val.shape)
# Display the shape of the validation target data.

history = model.fit(
# Start training the LSTM model and save the training history.

    X_train,
# Input data used for learning (previous 60-day sequences).

    y_train,
# Actual target values the model should predict.

    validation_data=(X_val, y_val),
# Use validation data to check performance on unseen data during training.

    epochs=20,
# Train using the entire training dataset 20 times.

    batch_size=32,
# Process 32 training samples at a time before updating weights.

    verbose=1
# Show training progress and results for each epoch.
)
```

Out [30]:

```
(2449, 60, 1)
(2449,)
(477, 60, 1)
(477,)
Epoch 1/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m4s[0m 26ms/step - loss: 0.0032 - val_loss: 6.8888e-04
```

```
Epoch 2/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 23ms/step - loss: 6.2498e-04 - val_loss: 4.2841e-04
Epoch 3/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 23ms/step - loss: 5.5333e-04 - val_loss: 3.9320e-04
Epoch 4/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 24ms/step - loss: 5.1948e-04 - val_loss: 5.4210e-04
Epoch 5/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 22ms/step - loss: 4.7930e-04 - val_loss: 3.3576e-04
Epoch 6/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 24ms/step - loss: 4.5601e-04 - val_loss: 0.0011
Epoch 7/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 23ms/step - loss: 4.0639e-04 - val_loss: 3.1385e-04
Epoch 8/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 23ms/step - loss: 3.7322e-04 - val_loss: 3.1227e-04
Epoch 9/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 24ms/step - loss: 3.9829e-04 - val_loss: 3.5047e-04
Epoch 10/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 24ms/step - loss: 3.8382e-04 - val_loss: 4.1895e-04
Epoch 11/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 26ms/step - loss: 3.6325e-04 - val_loss: 7.9173e-04
Epoch 12/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 27ms/step - loss: 3.2452e-04 - val_loss: 6.4627e-04
Epoch 13/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 27ms/step - loss: 3.0208e-04 - val_loss: 6.0145e-04
Epoch 14/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 28ms/step - loss: 2.9325e-04 - val_loss: 5.7639e-04
Epoch 15/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 29ms/step - loss: 2.9160e-04 - val_loss: 7.2252e-04
Epoch 16/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 26ms/step - loss: 2.6865e-04 - val_loss: 3.2313e-04
Epoch 17/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 26ms/step - loss: 2.5118e-04 - val_loss: 6.0937e-04
Epoch 18/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 26ms/step - loss: 2.7748e-04 - val_loss: 4.5616e-04
Epoch 19/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 29ms/step - loss: 2.5813e-04 - val_loss: 3.7433e-04
Epoch 20/20
[1m77/77[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m2s[0m 27ms/step - loss: 2.3237e-04 - val_loss: 2.5981e-04
```

In [31]:

```
#Evaluate performance on the test set using RMSE, MAE, and MAPE

#Step 1: Predict Test Values
predictions = model.predict(X_test)
# Generate predictions using the unseen test dataset.
```

Out [31]:

```
[1m15/15[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m1s[0m 25ms/step
```

In [32]:

```
#Step 2: Convert Back to Original Prices

predictions = scaler.inverse_transform(predictions)
# Convert scaled predictions back to actual Gold prices.

y_actual = scaler.inverse_transform(y_test.reshape(-1,1))
# Convert actual test values back to original NVIDIA prices.
```

In [33]:

```
#Step 3: Calculate RMSE, MAE, and MAPE

from sklearn.metrics import mean_squared_error
# Import function for Mean Squared Error.

from sklearn.metrics import mean_absolute_error
# Import function for Mean Absolute Error.

import numpy as np
# Import NumPy for calculations.


rmse = np.sqrt(mean_squared_error(y_actual, predictions))
```

```
# Calculate Root Mean Squared Error.

mae = mean_absolute_error(y_actual, predictions)
# Calculate Mean Absolute Error.

mape = np.mean(
    np.abs((y_actual - predictions) / y_actual)
) * 100
# Calculate Mean Absolute Percentage Error.


print("RMSE =", rmse)
# Show RMSE value.

print("MAE =", mae)
# Show MAE value.

print("MAPE =", mape)
# Show MAPE value as a percentage.
```

Out [33]:

```
RMSE = 28.712119475301325
MAE = 22.762239014181564
MAPE = 3.532619602561129
```

Req. 5: Plot 1:

The original time-series plot shows the movement of NVIDIA closing prices from January 2020 to July 2026. The series exhibits strong upward and downward trends, volatility, and several significant price fluctuations over time.

In [34]:

```
# Req. 5 (Plot original time series and actual vs predicted values for the test set)

import matplotlib.pyplot as plt
# Import plotting library.

plt.figure(figsize=(12,6))
# Create a larger chart.

plt.plot(df.index, df["Close"])
# Plot NVIDIA closing prices over time.

plt.title("NVIDIA Closing Price Time Series")
# Add chart title.

plt.xlabel("Date")
# Label x-axis.

plt.ylabel("Closing Price (USD)")
# Label y-axis.

plt.grid(True)
# Add grid for easier reading.

plt.show()
# Display the plot.
```

Out [34]:



Plot 2: Actual vs Predicted Values (Test Set)

In [35]:

```
import matplotlib.pyplot as plt
# Import plotting library.

plt.figure(figsize=(12,6))
# Create a larger figure.

plt.plot(y_actual, label="Actual", color="blue")
# Plot actual NVIDIA prices from the test set.

plt.plot(predictions, label="Predicted", color="red")
# Plot LSTM predicted prices.

plt.title("Actual vs Predicted NVIDIA Prices")
# Add chart title.

plt.xlabel("Time")
# Label x-axis.

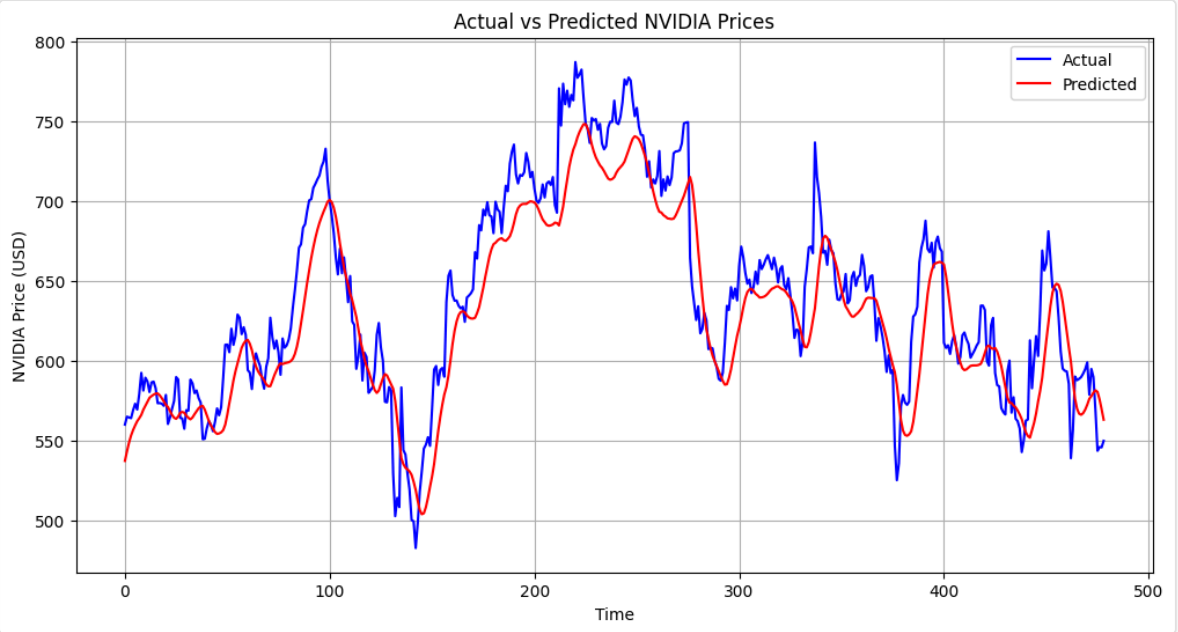
plt.ylabel("NVIDIA Price (USD)")
# Label y-axis.

plt.legend()
# Show labels for both lines.

plt.grid(True)
# Add grid.

plt.show()
# Display the chart.
```

Out [35]:



Req.6 Interpret the results and discuss how well the model captures trends and seasonality

RMSE (Root Mean Squared Error) = 3958.8041631011642 MAE (Mean Absolute Error) = 3019.776684085265 MAPE (Mean Absolute Percentage Error) = 3.5795266633705665 (predictions are only 3.58% wrong)

```
The average prediction error was approximately $3,020 (MAE), while the RMSE of $3,959 indicates some larger errors occurred but were not frequent. T
```

Using the model forecast the next 60 days price

In [36]:

```
# Forecast NVIDIA Prices for the Next 60 Days

import numpy as np
import matplotlib.pyplot as plt

# Store future predictions
future_predictions = []

# Take the most recent 60 days of scaled data
current_batch = scaled_data[-60:].reshape(1, 60, 1)

# Forecast the next 60 days
for i in range(60):

    # Predict next day's price
    next_prediction = model.predict(current_batch, verbose=0)

    # Save prediction
    future_predictions.append(next_prediction[0, 0])

    # Update the 60-day window
    current_batch = np.append(
        current_batch[:, 1:, :],
        [[[next_prediction[0, 0]]]],
        axis=1
    )

# Convert to NumPy array
future_predictions = np.array(future_predictions).reshape(-1, 1)

# Convert scaled values back to actual BTC prices
future_predictions = scaler.inverse_transform(future_predictions)
```

```
# Flatten for easier printing and plotting
future_predictions = future_predictions.flatten()

# Print forecasted prices
print("NVIDIA Price Forecast for the Next 60 Days:\n")

for day, price in enumerate(future_predictions, start=1):
    print(f"Day {day}: ${float(price):,.2f}")

# Plot forecast
plt.figure(figsize=(12, 6))

plt.plot(
    range(1, 61),
    future_predictions,
    marker='o',
    color='red',
    label='Forecasted BTC Price'
)

plt.title("NVIDIA Price Forecast for the Next 60 Days")
plt.xlabel("Days Ahead")
plt.ylabel("NVIDIA Price (USD)")
plt.grid(True)
plt.legend()

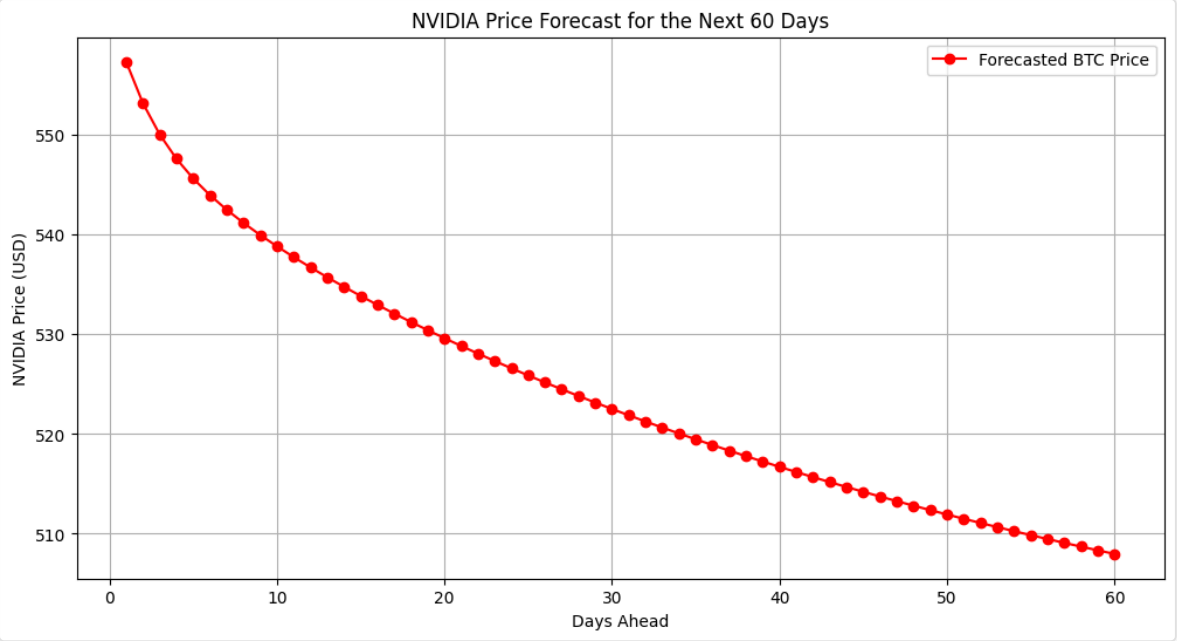
plt.show()
```

Out [36]:

```
NVIDIA Price Forecast for the Next 60 Days:

Day 1: $557.33
Day 2: $553.18
Day 3: $550.05
Day 4: $547.60
Day 5: $545.63
Day 6: $543.96
Day 7: $542.50
Day 8: $541.18
Day 9: $539.96
Day 10: $538.82
Day 11: $537.74
Day 12: $536.71
Day 13: $535.72
Day 14: $534.77
Day 15: $533.85
Day 16: $532.95
Day 17: $532.08
Day 18: $531.24
Day 19: $530.42
Day 20: $529.61
Day 21: $528.83
Day 22: $528.07
Day 23: $527.32
Day 24: $526.59
Day 25: $525.87
Day 26: $525.17
Day 27: $524.49
Day 28: $523.81
Day 29: $523.16
Day 30: $522.51
Day 31: $521.88
Day 32: $521.26
Day 33: $520.65
Day 34: $520.05
Day 35: $519.47
Day 36: $518.90
Day 37: $518.33
Day 38: $517.78
Day 39: $517.24
Day 40: $516.71
Day 41: $516.19
Day 42: $515.68
Day 43: $515.18
Day 44: $514.69
Day 45: $514.21
```

```
Day 46: $513.73
Day 47: $513.27
Day 48: $512.81
Day 49: $512.37
Day 50: $511.93
Day 51: $511.50
Day 52: $511.08
Day 53: $510.66
Day 54: $510.26
Day 55: $509.86
Day 56: $509.47
Day 57: $509.08
Day 58: $508.71
Day 59: $508.34
Day 60: $507.97
```



In [37]:

```python
#polting with previous data
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Create future dates
future_dates = pd.date_range(
    start=df.index[-1] + pd.Timedelta(days=1),
    periods=60,
    freq='D'
)

# Plot historical data and forecast
plt.figure(figsize=(15, 7))

# Last 180 days of actual BTC prices
plt.plot(
    df.index[-180:],
    df["Close"][-180:],
    label="Historical BTC Price",
    color="blue"
)

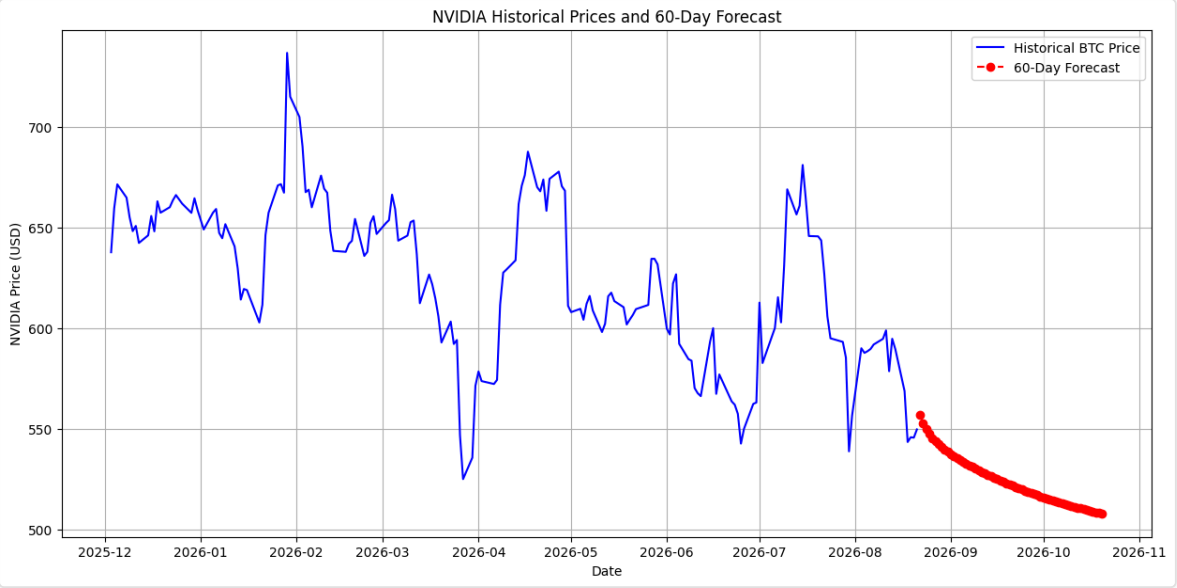
# Forecasted prices
plt.plot(
    future_dates,
    future_predictions,
    label="60-Day Forecast",
    color="red",
    linestyle="--",
```

```
    marker="o"
)

plt.title("NVIDIA Historical Prices and 60-Day Forecast")
plt.xlabel("Date")
plt.ylabel("NVIDIA Price (USD)")
plt.legend()
plt.grid(True)

plt.show()
```

Out [37]:



In [38]:

```
# Forecast NVIDIA Prices for the Next 60 Days

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Store future predictions
future_predictions = []

# Take the most recent 60 days of scaled data
current_batch = scaled_data[-60:].reshape(1, 60, 1)

# Forecast the next 60 days
for i in range(60):

    # Predict next day's price
    next_prediction = model.predict(current_batch, verbose=0)

    # Save prediction
    future_predictions.append(next_prediction[0, 0])

    # Update the 60-day window
    current_batch = np.append(
        current_batch[:, 1:, :],
        [[[next_prediction[0, 0]]]],
        axis=1
    )

# Convert to NumPy array
future_predictions = np.array(future_predictions).reshape(-1, 1)

# Convert scaled values back to actual NVIDIA prices
future_predictions = scaler.inverse_transform(future_predictions)
```

```
# Flatten array
future_predictions = future_predictions.flatten()

# Create future dates
last_date = df.index[-1]
future_dates = pd.date_range(
    start=last_date + pd.Timedelta(days=1),
    periods=60,
    freq="D"
)

# Create forecast dataframe
forecast_df = pd.DataFrame({
    "Date": future_dates,
    "Predicted Price": future_predictions
})

# Display date-wise predictions
print("\nNVIDIA Price Forecast for the Next 60 Days:\n")
print(forecast_df)

# Plot forecast with dates
plt.figure(figsize=(14, 6))

plt.plot(
    forecast_df["Date"],
    forecast_df["Predicted Price"],
    marker='o',
    linestyle='-',
    color='red',
    label='Forecasted NVIDIA Price'
)

plt.title("NVIDIA Price Forecast for the Next 60 Days")
plt.xlabel("Date")
plt.ylabel("NVIDIA Price (USD)")
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()
```

Out [38]:

```
NVIDIA Price Forecast for the Next 60 Days:

         Date  Predicted Price
0  2026-08-22       557.332031
1  2026-08-23       553.177856
2  2026-08-24       550.045898
3  2026-08-25       547.604980
4  2026-08-26       545.626282
5  2026-08-27       543.956726
6  2026-08-28       542.495544
7  2026-08-29       541.178162
8  2026-08-30       539.963257
9  2026-08-31       538.824524
10 2026-09-01       537.744995
11 2026-09-02       536.713867
12 2026-09-03       535.723511
13 2026-09-04       534.768799
14 2026-09-05       533.845825
15 2026-09-06       532.951294
16 2026-09-07       532.082825
17 2026-09-08       531.238220
18 2026-09-09       530.415771
19 2026-09-10       529.613953
20 2026-09-11       528.831299
21 2026-09-12       528.066956
22 2026-09-13       527.319702
23 2026-09-14       526.588623
24 2026-09-15       525.873291
25 2026-09-16       525.172852
26 2026-09-17       524.486816
```

```
27 2026-09-18       523.814697
28 2026-09-19       523.156067
29 2026-09-20       522.510681
30 2026-09-21       521.877930
31 2026-09-22       521.257690
32 2026-09-23       520.649414
33 2026-09-24       520.053223
34 2026-09-25       519.468445
35 2026-09-26       518.895081
36 2026-09-27       518.332886
37 2026-09-28       517.781433
38 2026-09-29       517.240479
39 2026-09-30       516.710083
40 2026-10-01       516.189758
41 2026-10-02       515.679382
42 2026-10-03       515.178772
43 2026-10-04       514.687683
44 2026-10-05       514.205933
45 2026-10-06       513.733337
46 2026-10-07       513.269531
47 2026-10-08       512.814575
48 2026-10-09       512.368286
49 2026-10-10       511.930237
50 2026-10-11       511.500458
51 2026-10-12       511.078674
52 2026-10-13       510.664734
53 2026-10-14       510.258545
54 2026-10-15       509.859802
55 2026-10-16       509.468475
56 2026-10-17       509.084320
57 2026-10-18       508.707306
58 2026-10-19       508.337280
59 2026-10-20       507.973877
```

